

1. Dados Simulados da Missão

Os dados usados no projeto foram criados através de inteligência artificial descrita a seguir pelo tópico 9, e está de acordo Seção 13 do trabalho da Global Solution, com geração, consumo e variáveis ambientais que se comportam de forma realista ao longo do dia.

A intenção foi montar um cenário com causa e efeito, em que a queda da turbina, a tempestade de poeira e a perda de comunicação se refletem nos números, e não apenas valores soltos.

Toda a telemetria fica organizada em quatro arquivos dentro da pasta data/, três em formato CSV e um em JSON, prontos para serem lidos diretamente pelo sistema.

1.1 Arquivos de Dados Utilizados

O [modulos_status.csv](#) traz consigo as situações operacionais dos oito módulos da base, sendo que para cada módulo há o nome, o status binário (1 ligado, 0 desligado), a criticidade declarada (normal, alerta ou crítico), a prioridade e uma descrição técnica curta. Sendo que é a partir disso que o arquivo reconhece se há algo de errado.

Já o [energia_leituras.csv](#) possui as séries temporais de energia ao longo dos oito horários do Sol, com a geração solar, a geração eólica, o consumo e a reserva da bateria em kWh e em porcentagem. Como a ordem das leituras importa para acompanhar a reserva caindo, esse arquivo é tratado como uma sequência cronológica.

E na [variaveis_ambientais.csv](#) reúne as condições do habitat e do ambiente externo nos mesmos oito horários (temperatura externa e interna, radiação, vento, qualidade da comunicação e pressão interna). Servindo como base para avaliar tanto o ambiente quanto a comunicação, e segue a mesma lógica de série temporal do arquivo de energia.

Por último, o [log_eventos.json](#) registra dez eventos em ordem cronológica, cada um com tipo, módulo afetado, severidade, descrição e ação tomada.

1.2 Status dos Módulos da Base

A base possui oito módulos no total, e cada um deles tem um status, uma criticidade e uma prioridade definida.

O que possui prioridade máxima é o “suporte_vida” (status 1, criticidade crítica, prioridade 1), que controla a pressurização do habitat, sendo basicamente o módulo que mantém a tripulação viva.

Logo após, vem os dois módulos de geração de energia. A “energia_solar” (status 1, criticidade crítica, prioridade 2) são os painéis fotovoltaicos, que durante o dia marciano carregam o peso da geração e a “energia_eolica” (status 0, criticidade alerta, prioridade 3) é a turbina, que está desligada por manutenção preventiva. Foi tirada de operação depois que o sistema acusou desgaste no rolamento.

Ja a comunicação (status 1, criticidade alerta, prioridade 4) é o canal com a Terra, e começa o dia já com o sinal ruim por causa de interferência atmosférica. O habitat (status 1, criticidade crítica, prioridade 2) cuida da estrutura pressurizada e do controle de temperatura lá dentro.

E por fim, os outros três são menos sensíveis, sendo o laboratório (status 1, criticidade normal, prioridade 5) é onde ficam os experimentos, e teve a prioridade rebaixada de propósito para economizar energia, o armazenamento (status 1, criticidade normal, prioridade 6) guarda mantimentos, peças e o que mais a missão precisa estocar e as baterias (status 1, criticidade crítica, prioridade 2) são o banco de 500 kWh, onde fica guardada toda a reserva de energia da base.

2.3 Inconsistência Proposital nos Dados

Conforme orientações da Global solution, no tópico 7 que tras os dados obrigatórios simulados é necessário: *“Ao menos uma inconsistência proposital nos dados para testar a capacidade de diagnóstico da equipe.”*

A inconsistência que foi plantada no dataset está no horário das 21:00, quando o “*energia_leituras.csv*” registra 18,5 kWh de geração solar. Isso é fisicamente impossível: às 21:00 já é noite marciana, a radiação no “*variaveis_ambientais.csv*” está zerada e, sem irradiância, os painéis não teriam como produzir energia.

O sistema pega essa inconsistência na função “*detectar_anomalias*”, que cruza os dois arquivos e se a radiação naquele horário é zero, ou seja, é noite, mas a geração solar é maior que zero, alguma coisa está errada e a leitura é marcada como anomalia.

A saída do sistema aponta exatamente o horário e o valor suspeito, sugerindo verificar o funcionamento dos sensores. Foi esse cruzamento de fontes diferentes que permitiu flagrar o erro, algo que não apareceria olhando só para o arquivo de energia isolado.

3. Estruturas de Dados Utilizadas

As estruturas de dados que foram utilizadas neste projeto foram escolhidas de acordo com a natureza de cada informação utilizada e o tipo de acesso necessário em cada situação.

3.1 Lista

A lista é uma sequência de estrutura, no qual foi ordenada que armazena elementos indexados por posição.

Neste projeto, ela foi usada para armazenar as séries temporais de geração solar, consumo e temperatura ao longo dos oito horários do sol marciano. A escolha se justifica porque a ordem dos elementos representa a própria sequência cronológica das leituras, permitindo percorrer os dados exatamente na ordem em que foram coletados.

Sendo utilizada no código da seguinte maneira, no módulo de [previsao.py](#):

```
# Prevê a reserva de energia do próximo ciclo a partir dos dados de telemetria
def prever_reserva_energetica(telemetria):
    # Extrai apenas a série da reserva de cada leitura (list comprehension)
    reservas = [item["reserva_bateria_pct"] for item in telemetria]
    # Cria os índices de tempo (0, 1, 2, ...) com o mesmo tamanho da série
    x = list(range(len(reservas)))
    # Calcula a reta de tendência da reserva
    m, b = regressao_linear(x, reservas)
    # Projeta a reserva para o próximo ciclo (o índice seguinte ao último)
    prox_ponto = prever_proximo_valor(m, b, len(reservas))

    # Retorna: histórico completo, previsão do próximo ciclo e tendência (m) m negativo indica reserva caindo; positivo indica subindo
    return reservas, prox_ponto, m
```

3.2 Dicionário

O dicionário é uma estrutura que armazena pares de chave e valor, dando o acesso direto a um dado a partir de sua chave.

Ele foi aplicado para acessar o status e a criticidade de cada módulo do sistema diretamente pelo nome, sem a necessidade de percorrer uma lista inteira, o que torna a consulta mais rápida e direta.

Sendo utilizada no código da seguinte maneira, no módulo de [logica.py](#):

```
#Funcao de avaliacao - Classifica um módulo como normal, alerta, crítico ou desconhecido.
def classificar_modulo(nome, telemetria):
    modulos = telemetria["modulos"]
    # Se o modulo nao existe na telemetria, retorna como desconhecido.
    if nome not in modulos:
        return "desconhecido"

    #Esta puxando os dados de um módulo específico de dentro da telemetria.
    info = modulos[nome]
    #Sendo que 1 significa ligado e 0 desligado
    status = info["status"]
    criticidade = info["criticidade"]
```

3.3 Fila (deque)

A fila segue o princípio FIFO (*First In, First Out*), onde o primeiro elemento inserido é também o primeiro a ser removido.

Neste projeto, foi utilizada para organizar os alertas pendentes por ordem de chegada, garantindo que o alerta mais antigo seja sempre tratado primeiro e respeitando, assim, a prioridade temporal dos eventos.

Sendo utilizada no código da seguinte maneira, no módulo de [logica.py](#):

```
#Fila: FIFO - First In First Out
class Fila:
    def __init__(self):
        # lista vazia que guarda os itens
        self.itens = []

    def enfileirar(self, item):
        # Item novo entra sempre no fim da fila.
        self.itens.append(item)

    def desenfileirar(self):
        # Remove e devolve o item mais antigo, o do inicio.
        if self.itens:
            return self.itens.pop(0)
        #Se vazia, devolve None.
        return None
```

3.4 Pilha

Já a pilha funciona de forma oposta, seguindo o princípio LIFO (Last In, First Out), em que o último elemento inserido é o primeiro a ser removido.

Ela foi empregada para registrar os eventos críticos analisados, permitindo consultar com facilidade o evento mais recente, o que é especialmente útil para a análise dos últimos acontecimentos do sistema.

Sendo utilizada no código da seguinte maneira, no módulo de [logica.py](#):

```
#Pilha LIFO - Last in First Out
class Pilha:
    def __init__(self):
        # pilha vazia que guarda os itens
        self.itens = []

    def empilhar(self, item):
        # Item novo vai para o topo.
        self.itens.append(item)
```

3.5 Matriz (lista de listas)

A ideia de matriz aparece na forma como as leituras de telemetria ficam organizadas por horário e por variável, em que cada horário funciona como uma linha e cada grandeza medida (radiação, geração solar, consumo, temperatura) como uma coluna. Na prática, essa estrutura bidimensional é representada por listas paralelas de leituras, a de energia e a de ambiente, percorridas pelo mesmo índice. Assim, o índice indica o horário (a linha) e os campos de cada leitura fazem o papel das colunas, o que permite cruzar grandezas diferentes de um mesmo momento.

Sendo utilizada no código da seguinte maneira, no módulo de [logica.py](#):

```
#Pilha LIFO - Last in First Out
class Pilha:
    def __init__(self):
        # pilha vazia que guarda os itens
        self.itens = []

    def empilhar(self, item):
        # Item novo vai para o topo.
        self.itens.append(item)
```

3.6 Hierarquia da Missão

Toda a telemetria da missão é agrupada num único dicionário aninhado, montado no [main.py](#) antes de seguir para o diagnóstico.

No nível de cima ficam três chaves, módulos, energia e ambiente. Sendo a chave do módulos é um dicionário em que cada nome de módulo aponta para outro dicionário com status, criticidade e prioridade.

As chaves energia e ambiente são listas de leituras, e cada leitura é por sua vez um dicionário com os valores daquele horário. Na prática, o bloco de energia carrega a geração solar, a geração eólica, o consumo e a reserva da bateria, enquanto o bloco de ambiente reúne o que diz respeito ao habitat, temperatura interna e externa, radiação, vento, qualidade da comunicação e pressão.

A combinação de dicionário (acesso por nome) com lista (ordem cronológica) num mesmo objeto foi o que permitiu passar toda a telemetria adiante de uma vez só, sem espalhar variáveis soltas pelo código.

4. Regras Lógicas de Diagnóstico

O diagnóstico da missão é montado a partir de uma série de regras que classificam cada subsistema em um de três estados, sendo eles normal, alerta ou crítico.

Essas regras usam a estrutura como IF, ELIF, ELSE para decidir o rótulo, combinada com os operadores lógicos AND, OR e NOT para expressar as condições.

4.1 Expressão Booleana Principal

Sendo a parte principal do diagnóstico, localizado em [logica.py](#), na função `expressao_booleana_principal`, que reduz todo o estado da missão a uma única expressão lógica.

Ela combina os rótulos dos subsistemas mais sensíveis e devolve True quando a base deve entrar em estado crítico:

```
#Se a situação for crítica o estado é "crítico".
if expressao_booleana_principal(telemetria):
    estado = "critico"
#se houver qualquer alerta pendente na fila, o estado é "alerta"
elif not fila_alertas.vazia():
    estado = "alerta"
#Caso contrario retorna como normal.
else:
    estado = "normal"

#Monta um dicionário
```

4.2 Regras e Ações do Diagnóstico

A lógica é sempre a mesma em todas as regras, o sistema detecta uma condição nos dados, define se é alerta ou crítico e já sugere o que fazer.

Começando pela energia, ela só vira crítica quando duas coisas acontecem juntas, a reserva abaixo de 25% e o consumo maior que a geração, sendo que a recomendação é desligar os módulos não essenciais e checar a geração. O alerta é mais simples, dispara quando a reserva cai abaixo de 50% ou quando já tem déficit com a reserva ainda abaixo de 60%, e pede pra ativar o modo economia.

Já na comunicação, se a qualidade do sinal cai abaixo de 30%, já é crítico, e a ação é abrir um canal alternativo. E entra em alerta, quando não é verdade que a qualidade está em 70% ou mais e a recomendação é olhar as antenas e transmissores.

E no ambiente a gente cruza radiação com temperatura, sendo crítico quando a radiação chega a 0,6 mSv e a temperatura está fora dos 18 a 26 °C ao mesmo tempo, pedindo para verificar a radiação e o isolamento do habitat. Se só uma das duas aparece, radiação a partir de 0,3 mSv ou temperatura fora da faixa, é só alerta e a ação é uma inspeção preventiva.

Os módulos também são olhados um por um e quando um módulo crítico aparece desligado, o estado é crítico e o sistema aciona o protocolo daquele módulo.

As duas últimas regras saem da previsão, onde se a reserva projetada para o próximo ciclo fica abaixo de 25%, é crítico e a recomendação é cortar o consumo dos não essenciais e se fica abaixo de 50% mas ainda com tendência de queda, é alerta, e aí a ação é otimizar o uso da reserva.

4.3 Explicação das Regras em Linguagem Simples

Regra 1 - Energia crítica (uso de AND)

Esta regra avalia o subsistema de energia e usa o operador AND para exigir que duas coisas ruins aconteçam ao mesmo tempo, onde o sistema só declara a energia como crítica quando a reserva da bateria está abaixo de 25% E (**AND**) o consumo está maior que a geração.

A lógica é que reserva baixa, sozinha, ainda é administrável se a base estiver gerando mais do que gasta, o que realmente assusta é estar com pouca reserva e ainda por cima perdendo carga e se tiver essas duas condições a recomendação gerada é desligar os módulos não essenciais e verificar os sistemas de geração.

Regra 2 - Comunicação em alerta (uso de NOT)

A avaliação da comunicação usa o operador NOT de forma explícita para tratar a faixa intermediária do sinal.

Primeiro o sistema verifica se a qualidade está abaixo de 30%, o que já configura estado crítico e se não for tão baixa assim, entra a regra com NOT.

Se **NÃO (NOT)** for verdade que a qualidade está em 70% ou mais, ou seja, se o sinal está na faixa entre 30% e 70%, o estado vira alerta e acima de 70% a comunicação é considerada normal e em alerta, a recomendação é verificar antenas e transmissores; em estado crítico, estabelecer um canal de comunicação alternativo.

Regra 3 - Risco indireto: energia em alerta com comunicação crítica (uso de OR e AND)

A expressão booleana principal eleva a base a estado crítico se a energia, o ambiente ou o suporte à vida estiverem críticos (várias condições ligadas por **OR**), OU se a energia já estiver em alerta e a comunicação estiver crítica ao mesmo tempo.

Esse último trecho com AND modela uma situação de risco indireto, cada variável sozinha não está num valor extremo, mas a combinação de uma base perdendo energia e ficando sem contato com a Terra é justamente o pior momento para um imprevisto, porque a tripulação fica isolada sem conseguir pedir apoio.

5. Alertas Automáticos

O sistema os gera automaticamente os alertas a partir do diagnóstico de cada subsistema e de cada módulo, quando uma avaliação devolve alerta ou crítico, o nome correspondente entra

na fila de alertas pendentes, que segue a ordem de chegada (FIFO), de modo que o primeiro problema detectado é o primeiro a aparecer na lista.

Os itens marcados como críticos, além de entrarem na fila, são empilhados numa pilha separada (LIFO), o que permite consultar com facilidade o evento crítico mais recente.

Os eventos críticos vindos do log histórico também são empilhados, com um prefixo que os distingue dos atuais. Assim, a priorização acontece sozinha, a fila preserva a cronologia dos alertas e a pilha guarda o topo do que há de mais grave.

5.1 Níveis de Severidade

O sistema trabalha com três níveis de severidade, sendo eles:

- O nível **CRÍTICO**: acionado quando algum subsistema vital falha de forma grave, ou seja quando a energia crítica (reserva abaixo de 25% com déficit de geração), ambiente crítico (radiação alta junto de temperatura fora de faixa), suporte à vida desligado, ou a combinação de energia em alerta com comunicação crítica. É o nível que exige ação imediata da tripulação.
- O nível **ALERTA**: cobre as situações intermediárias, em que algo já saiu do normal mas ainda há margem para agir, acontece quando a reserva abaixo de 50%, sinal de comunicação entre 30% e 70%, radiação ou temperatura fora da faixa segura, ou um módulo de menor criticidade desligado.
- O nível **NORMAL**: é o estado de rotina, atribuído quando nenhuma das condições de alerta ou crítico se confirma, sendo o estado global da missão recebe o rótulo mais grave aplicável e se a expressão booleana principal aponta situação crítica, o estado é crítico; se não, mas existe qualquer item na fila de alertas, o estado é alerta; caso contrário, normal.

6. Análise e Previsão de Dados

6.1 Variável Analisada e Justificativa

A variável que foi escolhida pra prever foi a reserva da bateria em porcentagem (reserva_bateria_pct), acompanhada ao longo dos oito horários do Sol marciano. E não foi uma escolha aleatória, numa base isolada como a Aurora, a reserva da bateria é o que segura os módulos críticos de pé justamente quando não tem geração, tipo durante a longa noite marciana.

Se essa reserva cai abaixo de um limite seguro, o suporte à vida já entra em risco. Por isso ela é a variável que mais vale a pena antecipar. Prever pra onde a reserva está indo permite agir antes do problema virar crítico, em vez de só correr atrás depois que a bateria já esvaziou.

6.2 Técnica Utilizada

A técnica foi regressão linear pelo método dos mínimos quadrados, feita do zero no previsao.py, sem Pandas, NumPy ou Scikit-learn. A ideia é achar a reta $y = a \cdot x + b$ que melhor descreve pra onde a reserva está indo ao longo do tempo, onde x é o índice do horário e y é a reserva da bateria naquele momento.

Primeiro o sistema monta a série de reservas e o vetor de tempo. Depois calcula quatro somatórias:

- a soma dos x ;
- a soma dos y ;
- a soma dos produtos $x \cdot y$;
- a soma dos x ao quadrado.

Com essas somatórias, ele acha o coeficiente angular pela fórmula $a = (nSoma_{xy} - Soma_x Soma_y) / (nSoma_{x^2} - Soma_x^2)$ e o intercepto $b = (Soma_y - aSoma_x) / n$. A previsão do próximo ciclo é só aplicar a reta no índice seguinte ao último.

Como comparação, o mesmo arquivo tem a função `media_movel`, que faz uma estimativa rápida de curto prazo usando a média dos últimos valores da série. Ela serve de contraponto, enquanto a regressão pega a tendência do dia todo, a média móvel responde mais ao que aconteceu por último.

6.3 Resultado e Impacto na Decisão

Com os dados do Sol 147, a série de reservas ficou em [56,0; 52,9; 50,4; 49,5; 50,9; 49,3; 45,8; 45,6], e a regressão devolveu um coeficiente angular de - 1,3119 pontos percentuais por ciclo. Ou seja, a reserva vem caindo em média 1,3 ponto a cada horário. Aplicando a reta ao próximo ciclo, a projeção é de 44,1%.

Ele mexe direto numa recomendação do sistema, como os 44,1% previstos cruzam o limiar de alerta de 50% e a tendência é de queda, o módulo de recomendações dispara a ação "otimizar uso da reserva energética". É exatamente o que o enunciado pedia, a previsão entra na lógica de decisão e gera uma ação preventiva antes da reserva chegar num nível perigoso. Se a queda projetada fosse mais feia, abaixo de 25%, a recomendação subiria pra reduzir o consumo dos módulos não essenciais.

7. Recomendações Geradas pelo Sistema

Tudo vem da função `gerar_recomendacoes`, que cruza o diagnóstico já calculado com a previsão de energia e devolve uma lista ordenada por prioridade. A ordem em que as regras são checadadas define a urgência, o suporte à vida vem sempre primeiro, depois os subsistemas (energia, ambiente e comunicação), e por último as recomendações que dependem da projeção da reserva. Se nenhuma condição é satisfeita, o sistema devolve uma mensagem padrão indicando operação normal. Abaixo estão as recomendações organizadas pelos três níveis de prioridade.

Com o dataset do Sol 147, o sistema gerou duas recomendações de prioridade alta.

7.1 Prioridade Crítica

Acionadas quando um subsistema vital está crítico. São as ações de resposta imediata:

- **Suporte à vida crítico:** acionar protocolo de suporte à vida.
- **Energia em estado crítico:** desligar módulos não essenciais e verificar sistemas de geração.
- **Condições ambientais críticas:** verificar radiação e isolamento do habitat.
- **Canais de comunicação comprometidos:** estabelecer canais de comunicação alternativos.
- **Reserva projetada abaixo de 25%:** reduzir consumo dos módulos não essenciais.

No dataset do Sol 147 nenhuma recomendação crítica foi disparada, já que o estado global ficou em alerta e não em crítico.

7.2 Prioridade Alta

Acionadas em estado de alerta, quando ainda há margem para agir de forma preventiva:

- **Energia em estado de alerta:** ativar modo economia de energia.
- **Condições ambientais perigosas:** realizar inspeção preventiva do habitat.
- **Falhas nos sistemas de comunicação:** verificar antenas e transmissores.
- **Reserva projetada abaixo de 50% com tendência negativa:** otimizar uso da reserva energética.

Foram essas duas as recomendações geradas no Sol 147: ativar o modo economia, por causa da energia em alerta, e otimizar o uso da reserva, por causa da previsão de 44,1% com tendência de queda.

Prioridade Normal

Quando nenhuma condição de alerta ou crítico se confirma, o sistema retorna a recomendação padrão de rotina:

- Sistemas operando normalmente: manter monitoramento de rotina.